LLM-as-a-Judge Reliability

Combined Presentation: Benchmarks, Biases, and Robustness

Parsa Gholami, Aria Alyasin, Mohsen Shirazi

Sharif University of Technology

Category 1: Benchmarks & Frameworks for Judges

Paper 1:

LLMs instead of Human Judges?

A Large Scale Empirical Study across 20 NLP Evaluation Tasks

The Problem: Can LLMs Replace Human Judges?

- Human evaluation is informative, but **slow, expensive, and difficult to scale**.
- LLM-as-a-Judge is increasingly used as a cheaper automatic substitute.
- Prior evidence is mixed and often based on **few datasets or proprietary models**.

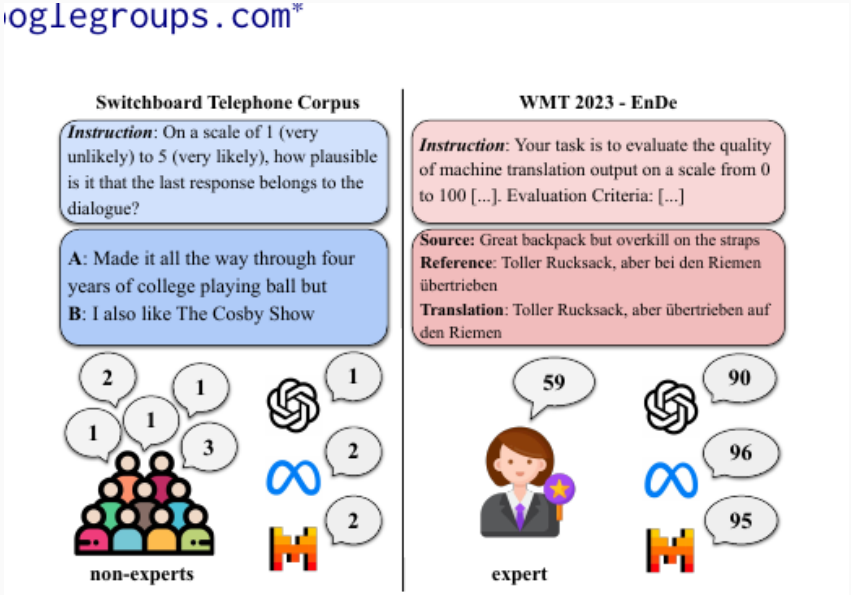


Figure 1: Evaluation by expert and non-expert human annotators and by LLMs for two tasks involving human-generated (left) and machine-generated text (right).

Core research question:

How reliably can current LLMs reproduce human judgments?

Figure 1 from Bavaresco et al. (2025): examples of human and LLM evaluation on human- and machine-generated text.

Main Contribution: JUDGE-BENCH

- **20 NLP datasets** with existing human annotations and **70,000+ test instances**.
- **11 LLM judges**: proprietary and open-weight models, from 7B-scale models to GPT-4o.
- Deliberately varies four important axes:
  1. **What is judged?** Grammar, toxicity, coherence, factual consistency, verbosity, translation quality, reasoning traces, etc.
  2. **Annotation type:** categorical vs. graded scores.
  3. **Who judged it?** experts vs. non-experts.
  4. **Text source:** human-written vs. model-generated.

Novelty
Instead of asking whether one judge works on one benchmark, the paper asks whether **LLM judging is reliable across tasks, properties, annotator expertise, and data sources**.

How Are the LLM Judges Evaluated?

- The authors reuse the **original human annotation instructions** as prompts whenever available.
- Model outputs are constrained to the expected label/score format:
  "Answer with one of {}. Do not explain your answer."
- **Categorical annotations** $\rightarrow$ Cohen's κ.
- **Graded annotations** $\rightarrow$ Spearman's ρ.
- A human **upper bound** is estimated by comparing individual raters with aggregated human judgments.
- CoT, few-shot prompts, and prompt paraphrases are also tested, but **do not produce systematic improvements**.

Interpretation
High agreement means an LLM reproduces the human annotation pattern for that *specific task*; it does not establish a universally good judge.

Result 1: There Is No Universal LLM Judge

	GPT-4o	Llama-3.1-70B	Mixtral-8x22B	Gemini-1.5
Avg. categorical κ	0.28	0.28	0.24	0.22
Avg. graded ρ	0.50	0.43	0.44	0.43

- GPT-4o performs best in several settings, but **open models are often close and sometimes better**.
- Example: on categorical CoLa, Mixtral-8x7B reaches $\kappa = 0.55$ vs. GPT-4o's 0.34.
- On SummEval graded quality, Mixtral-8x22B reaches $\rho = 0.54$ vs. GPT-4o's 0.35.
- **No single model dominates every evaluation property.**

Values reproduced from Table 1 of Bavaresco et al. (2025).

Result 2: LLMs Agree More with Non-Experts

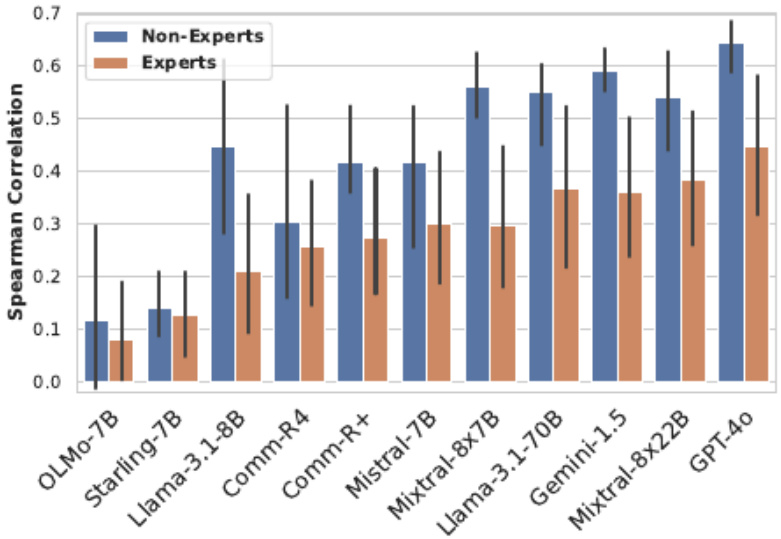


Figure 2: Average model correlation with human experts vs. non-experts in datasets with graded annotations.

- For graded annotations, **every evaluated model** aligns better with non-expert than expert judgments.
- One possible explanation: non-experts may rely more on surface-level cues, while experts apply stricter domain-specific criteria.
- The authors explicitly call this explanation **speculative**.

Implication: "Agreement with humans" depends on **which humans**.

Result 3: Judging Model Outputs Is Harder

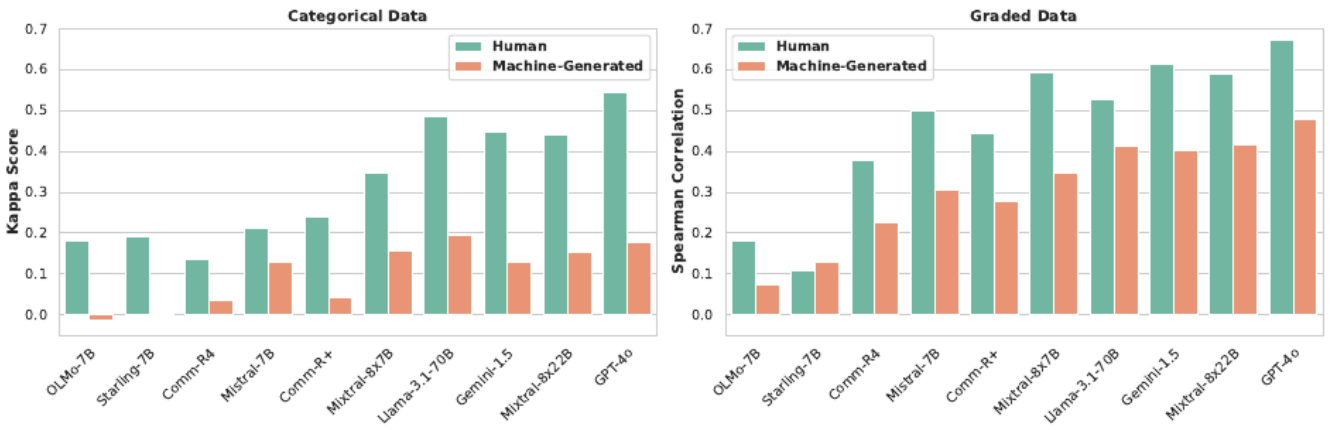


Figure 4: Scores (Cohen's κ for categorical annotations and Spearman's correlation for graded annotations) on test items involving human language vs. machine-generated outputs.

- Across both categorical and graded annotations, models show **better human alignment on human-written language**.
- Alignment drops when evaluating **machine-generated outputs**—exactly the setting where LLM-as-a-Judge is often deployed.
- The paper connects this trend to prior evidence of possible bias toward model-generated text.

Takeaways, Limitations, and the Bridge to JETTS

What the paper establishes

- LLM judges are **useful on some tasks**, including instruction following and mathematical reasoning traces.
- Reliability varies by **task, property, human expertise, and text source**.
- Recommendation: **validate and calibrate** an LLM judge against task-specific human annotations before deployment.

Important limitations

- Human–LLM agreement can be misleading if both share similar biases.
- Existing datasets may have quality, representativeness, or leakage issues.
- Mostly English; pairwise preference judging is left for future work.

Transition to JETTS

This paper asks: **"Does the judge reproduce human judgments?"**

JETTS asks the next question: **"Is the judge actually useful inside test-time scaling?"**

Paper 2:

Evaluating Judges as Evaluators

The JETTS Benchmark of LLM-as-Judges as Test-Time Scaling Evaluators

Motivation: Evaluation Becomes Part of Inference

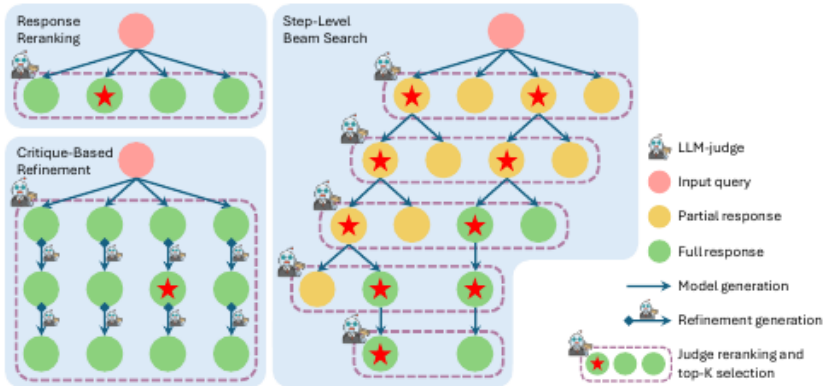
- Test-time scaling gives a generator LLM extra compute during inference.
- This usually needs an **evaluator**: choose the best sample, guide a search tree, or critique a draft.
- Reward models are common, but LLM judges are attractive because they are **instructable** and can produce **natural-language critiques**.

Central question
Are LLM judges actually useful evaluators when they are placed inside test-time scaling algorithms?

Why this matters: a judge that looks good on a static benchmark may still fail when it must improve a generator's final output.

Main Contribution: JETTS

- First systematic benchmark for using LLM judges as **test-time scaling evaluators**.
- Evaluates **10 judge models** from 7B to 70B parameters.
- Uses **8 generator models** from 6.7B to 72B parameters.
- Covers three domains: **math**, **code**, and **instruction following**.



Judge	Reranking ↑	Beam Search ↑	Refinement ↑
Prometheus-2 7B	-0.107	-0.100	0.976
SFR-Judge 8B	0.035	0.001	0.941
Skywork-Critic 8B	0.039	0.046	-
OffsetBias 8B	-0.020	0.015	-
Themis 8B	-0.149	-0.021	0.996
SFR-Judge 12B	0.013	0.047	0.943
Prometheus-2 8x7B	-0.079	-0.083	0.966
SFR-Judge 70B	0.171	0.138	0.951
Skywork-Critic 70B	0.177	0.132	-
Self-Taught Eval. 70B	0.095	0.072	-
Best RM	0.113	-	-
Best PRM	-	0.195	-
Random	-0.193	-0.139	-
Baseline Value	0.000	0.000	1.000

Figure 1 from Zhou et al. (2025): three JETTS tasks and aggregate leaderboard.

What Does JETTS Measure?

- JETTS does **not** only ask whether the judge agrees with labels.
- It asks whether judge-guided computation improves the generator relative to simpler baselines.

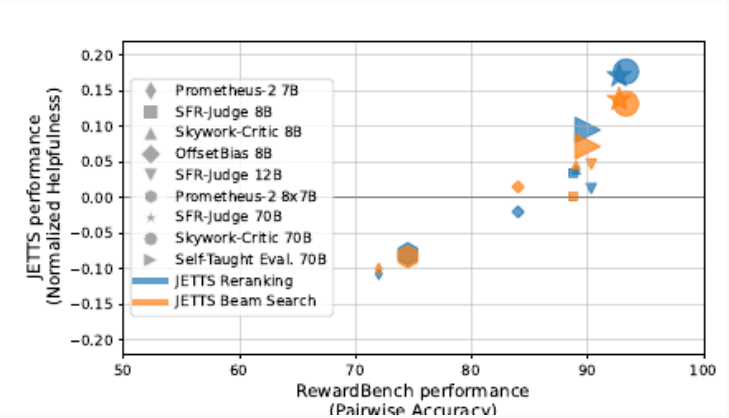
Normalized helpfulness for reranking and beam search

$$h = \frac{p_{\text{judge}} - p_{\text{greedy}}}{p_{\text{oracle}} - p_{\text{greedy}}}$$

- $h = 0$: no better than greedy decoding.
- $h < 0$: judge actively hurts performance.
- $h = 1$: oracle selection among available candidates.

Task 1: Response Reranking

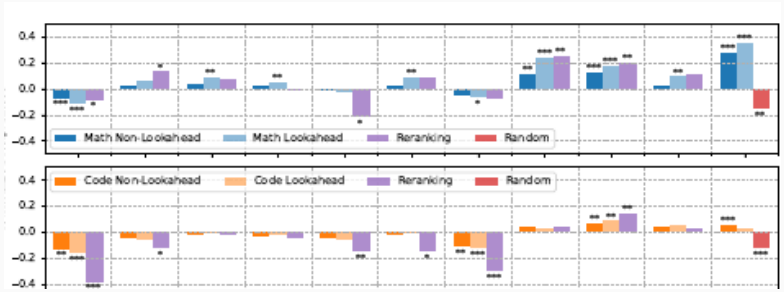
- Generate one greedy response and nine sampled responses.
- Judges select the best candidate by:
  - pairwise round-robin comparison, or
  - single-response rating.
- **Main result:** pairwise reranking is much stronger than single-rating.
- Judges are competitive with outcome reward models, especially for instruction following.



Important nuance
RewardBench can make small judges look close to large judges, but JETTS reveals a stronger scale gap in actual test-time scaling use.

Task 2: Step-Level Beam Search

- The generator produces candidate **next steps**; the judge keeps the most promising paths.
- JETTS uses a $(10, 2)$ beam search with depth up to 10.
- This is harder than reranking because the judge must assess **unfinished partial responses**.

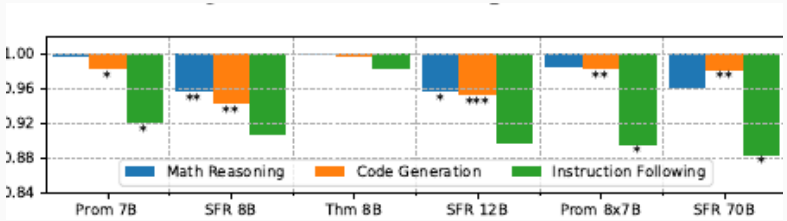


Main result:

- LLM judges lag behind a small process reward model.
- Larger judges help math and instruction-following more than code.

Task 3: Critique-Based Refinement

- Start from a seed response.
- Judge gives a score and natural-language critique.
- Generator revises the answer; the process repeats for 9 rounds.
- Final responses are reranked.



Main result:

- No judge yields beneficial refinement across task categories.
- Critiques often emphasize style over correctness and do not reliably fix errors.

Final Takeaways and Link to the Previous Paper

Key findings

- LLM judges can help with **response reranking**.
- They are weaker than **process reward models** for step-level search.
- Current critiques are not reliable tools for response refinement.

Limitations and implications

- Better static evaluation scores do not guarantee better test-time scaling utility.
- Coding remains especially difficult for judge-guided scaling.
- Future judges need better reasoning, not only better final verdicts.

Connection to "LLMs instead of Human Judges?"
The first paper asks: **Do LLM judges agree with humans?**

JETTS asks: **Can LLM judges improve generation when used as evaluators during inference?**

Category 2: Bias Quantification & Fairness Analysis

Paper 3:
An Empirical Analysis of Uncertainty in Large Language Model Evaluations

Category Alignment & Positioning

- **Taxonomy Alignment:** Category 2: Bias Quantification & Fairness Analysis
- **Beyond Static Prejudices:** While prior literature focuses heavily on static evaluator biases (e.g., length, position, and brand preference), this work explores **dynamic stability**.
- **Internal States:** Shifting focus from verbalized outputs to internal logit probability distributions of the evaluator.
- **Systemic Failure:** Demonstrating how hidden evaluator uncertainty leads to fragile, non-reproducible model rankings.

Paper Overview & Core Goals

- **The Core Problem:** Automated LLM judges are widely used to replace human raters, yet their internal evaluation confidence remains unquantified and volatile.
- **The Dual-Uncertainty Paradigm:** Real-world evaluations depend on a complex interaction between the judge's certainty and the candidate's generation certainty.
- **Three-Step Research Agenda:**
  1. **Existence:** Measuring raw logit probabilities to prove uncertainty exists across major model families.
  2. **Mitigation:** Testing prompt formatting styles to stabilize decision boundaries.
  3. **Utilization:** Training an uncertainty-aware judge to spot hallucinations in out-of-distribution (OOD) scenarios.

The Dual-Uncertainty Interaction

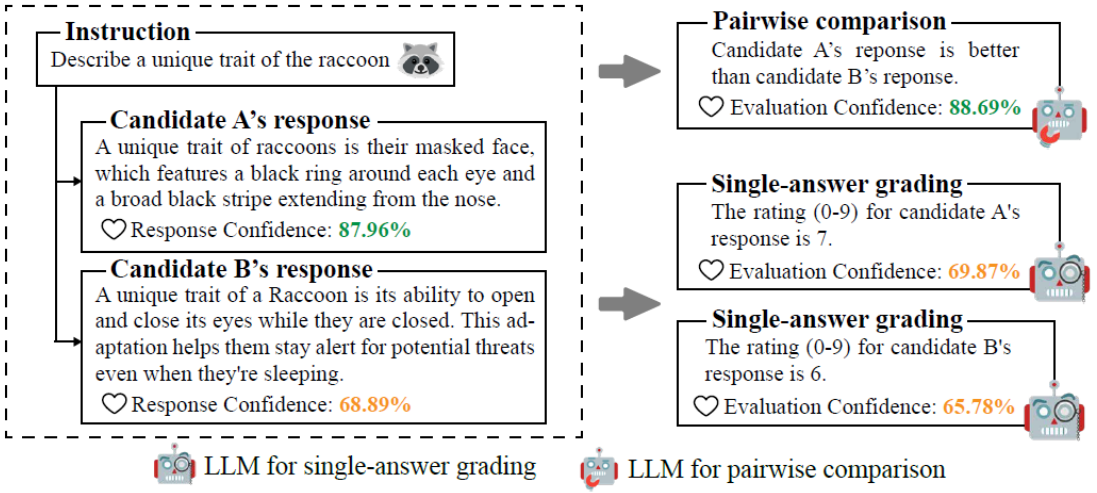


Figure 1: An example of uncertainty (i.e., model confidence) in model-based LLM evaluation. The evaluation process is influenced by the uncertainty of both the evaluator and the candidate model.

Figure 1: Visualizing the interplay of Candidate Response Confidence and Evaluator Judge Confidence.

Part 1: Quantifying Uncertainty (Existence)

- **Logit-Based Probabilities:** Instead of prompting judges to verbalize confidence (which suffers from poor numeric calibration), this work extracts the raw logits directly at the last layer of generation.
- **Evaluation Confidence:** The precise probability of the specific token representing the final decision (e.g., the score "7" in absolute grading, or "1", "2", "Tie" in pairwise comparison).
- **Response Confidence:** Calculated as the mathematical average of the generation probabilities of all tokens in the candidate's output.
- **Zero-Cost Metadata:** Extracted directly during inference, bypassing the need for computationally expensive multi-sample consistency checks.

Part 2: Mitigating Uncertainty (Mitigation)

- **Format-Induced Doubt:** Default templates prompt judges to output the score first. This forces an immediate decision under high attention entropy, dispersing the probability mass.
- **Chain-of-Thought (CoT) Calibration:** Forcing the judge to generate reasoning before the verdict token acts as a natural logit stabilizer, concentrating probability mass on the final score token.
- **Self-Generated References:** Instructing the judge to write its own reference answer first provides a solid comparative anchor, reducing rating volatility.
- **Post-Training Stabilization:** Specialized evaluator models (e.g., Prometheus2) trained in CoT formats display remarkably high confidence, but suffer from sensitivity to distribution shifts.

Part 3: Utilizing Uncertainty (ConfiLM)

- **The Ingestion Concept:** Candidate models express lower token probabilities when attempting to hallucinate facts. By feeding this uncertainty into the judge, the judge can detect errors.
- **The Verbalization Bridge:** LLMs struggle with continuous numerical float values in prompts. The authors map candidate logits into discrete natural language statements (e.g., "Highly confident", "Neutral", "Clearly doubtful").
- **ConfiLM Fine-Tuning:** Fine-tuning Llama-3-8B-Instruct on Alpaca data augmented with these verbalized confidence statements.
- **The Olympic 2024 Dataset:** A manually curated, 220-instance out-of-distribution (OOD) testing suite annotated by PhD-level experts (97.27% agreement) to evaluate real-time factual robustness.

Part 3: Utilizing Uncertainty (ConfiLM)

- **The Ingestion Concept:** Candidate models express lower token probabilities when attempting to hallucinate facts. By feeding this uncertainty into the judge, the judge can detect errors.
- **The Verbalization Bridge:** LLMs struggle with continuous numerical float values in prompts. The authors map candidate logits into discrete natural language statements (e.g., "Highly confident", "Neutral", "Clearly doubtful").
- **Confidence Buckets:** Ten evenly-spaced ranges from [0.0, 0.1) $\rightarrow$ *"Complete doubt"* up to [0.9, 1.0] $\rightarrow$ *"Absolute confidence"*, with *"Neutral"* at the midpoint [0.5, 0.6).
- **ConfiLM Fine-Tuning:** Fine-tuning Llama-3-8B-Instruct on Alpaca data augmented with these verbalized confidence statements.
- **The Olympic 2024 Dataset:** A manually curated, 220-instance out-of-distribution (OOD) testing suite annotated by PhD-level experts (97.27% agreement) to evaluate real-time factual robustness.

Multi-Phase Experimental Pipeline

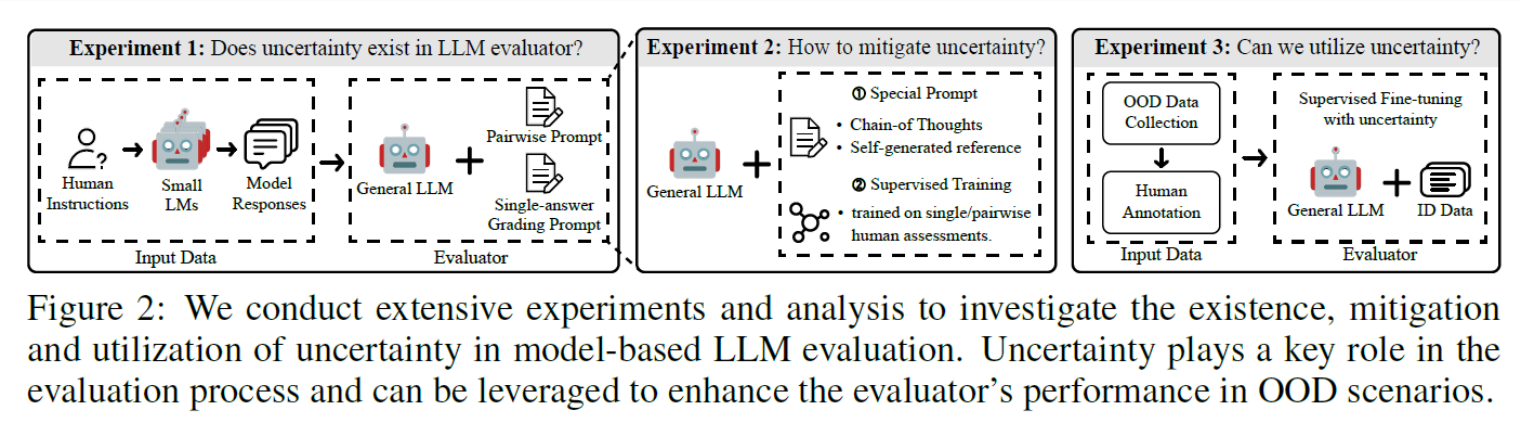


Figure 2: We conduct extensive experiments and analysis to investigate the existence, mitigation and utilization of uncertainty in model-based LLM evaluation. Uncertainty plays a key role in the evaluation process and can be leveraged to enhance the evaluator's performance in OOD scenarios.

Figure 2: The three sequential experimental phases: Existence, Mitigation, and Utilization.

Key Empirical Findings

- **Single-Answer vs. Pairwise Gap:** Absolute grading forces judges to evaluate in isolation, leading to high internal doubt. Pairwise comparisons provide relative anchors, resulting in highly decisive and confident assessments.
- **Self-Preference Bias:** Evaluations within the same model family (e.g., Llama assessing Llama) result in artificially inflated ratings paired with unwarranted confidence spikes due to shared pre-training and stylistic overlap.
- **The Capability Paradox:** General capability on reasoning benchmarks does not prevent evaluation doubt. Highly capable models like GPT-4o exhibit high uncertainty in default grading formats compared to less advanced models.

ConfiLM OOD Performance

- **OOD Challenges:** Standard judges fail in highly specific OOD domains (like real-time sports results) because they rely on in-distribution training data and overlook factual inaccuracies.
- **The "Rugby Sevens" Case Study:** When a candidate model generated a news report with a fabricated date, GPT-4o and standard fine-tuned models chose the hallucinated, verbose response.
- **The ConfiLM Victory:** By ingesting the verbalized low-confidence of the candidate, ConfiLM correctly flagged the response as inaccurate and penalized the hallucination.

Core Framework Strengths

- **Dual-Uncertainty Interface:** Establishes the first unified framework integrating candidate generation doubt with judge decision confidence.
- **Zero-Cost Calibration:** Demonstrates that Chain-of-Thought (CoT) acts as a computationally free, inference-time logit stabilizer.
- **The Verbalization Bridge:** Successfully bypasses the numerical limits of language models by translating float values into natural adjectives.
- **Expert-Curated Benchmark:** The Olympic 2024 dataset provides a rigorous, highly aligned OOD testbed for automated judges.

Operational Limitations

- **Closed-Source API Barrier:** Logit extraction is impossible on proprietary commercial platforms (such as Claude and Gemini) that withhold token probabilities.
- **Inference Latency Overhead:** Forcing a detailed explanation before outputting the score token elevates generation costs and runtime latency.
- **Exposure Bias in SFT:** Specialized judges fine-tuned on rigid templates suffer from high scoring volatility when exposed to unfamiliar domains.

Critical Strategic Outlook

- **Confidence-Triggered Escalation:** Future AI-evaluation platforms should establish automated confidence thresholds, triggering selective escalation to human raters for low-confidence judgments.
- **Multimodal Uncertainty:** Extending the framework to measure uncertainty across image, audio, and video inputs in multi-modal judges.
- **Multi-Turn Conversation Decay:** Investigating how evaluation uncertainty propagates or decays across complex, multi-turn dialogues.

Paper 4:
Preference Leakage: A Contamination Problem in LLM-as-a-Judge

Context: The Evolution of Model Evaluation

- **Shift in Evaluation Paradigms**:
  - Traditional lexical n-gram metrics (BLEU, ROUGE) fail to capture semantic nuances in open-ended LLM outputs.
  - **LLM-as-a-Judge** has emerged as the industry standard for scalable, fast, and human-aligned model benchmarking.
- **The Synthetic Data Engine**:
  - Modern LLM alignment relies heavily on synthetic datasets generated by state-of-the-art teacher models (e.g., GPT-4o).
- **The Core Vulnerability**:
  - When the model producing the training data and the model judging the student share architectural or family ties, systemic evaluation bias occurs.

Category Alignment: Data vs. Preference Leakage

- **Data Leakage (Classical Contamination)**:
  - Direct overlap between pre-training/fine-tuning datasets and evaluation benchmark test sets.
  - Model simply memorizes specific ground-truth questions and answers.

- **Preference Leakage (Evaluator Contamination)**:
  - Structural, familial, or distillation relatedness between Data Generator (M_G) and Judge (M_J).
  - Preferences are transferred to Student (M_S) via synthetic training data (D_{syn}).
  - Evaluator inflates scores based on inherited stylistic and structural priors rather than objective response correctness.

Overview Diagram: Preference vs. Data Leakage

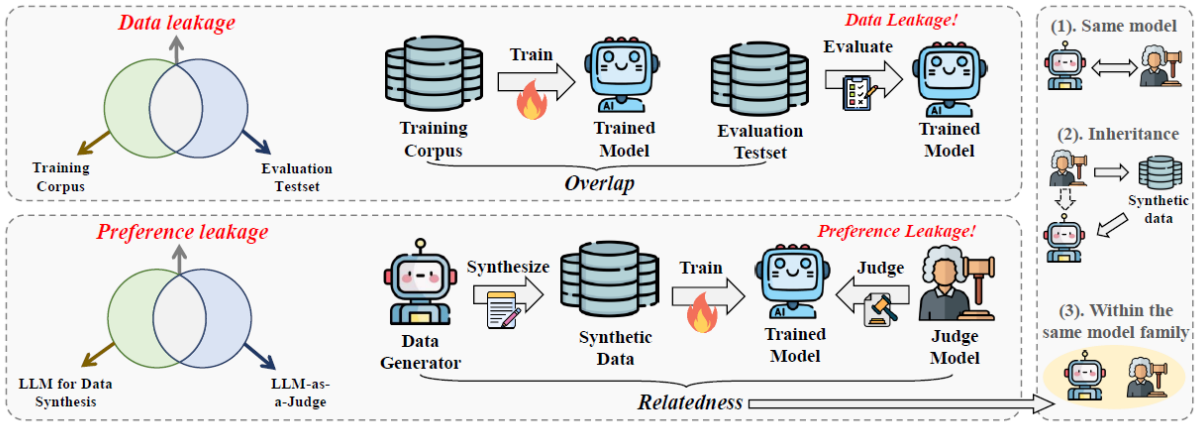


Figure 1: Overview of preference leakage. We make a comparison between data leakage and preference leakage and present three types of relatedness: being the same model, having an inheritance relationship and belonging to the same model family.

Figure 3: Conceptual Architecture of Preference Leakage and Model Lineage Relationships

Methodology & Experimental Pipeline

End-to-End Experimental Pipeline

- **Stage 1: Seed Sampling & Prompt Extraction**:
  - Sampled 30,000 diverse open-ended prompts across diverse domains from UltraFeedback.
- **Stage 2: Synthetic Data Generation ($M_G \rightarrow D_{syn}$)**:
  - Teacher models (M_G) generate high-quality candidate responses to form instruction-tuning datasets.
- **Stage 3: Student Model Post-Training (M_S)**:
  - Base models (Mistral-7B, Qwen-2.5-14B) are trained on D_{syn} via SFT, DPO, or In-Context Learning (ICL).
- **Stage 4: Pairwise Evaluation & PLS Calculation**:
  - Judge models (M_J) evaluate M_S against baseline models on Arena-Hard and AlpacaEval 2.0 to quantify score inflation.

Part 1: Problem Definition & Lineage Taxonomy

- **Conceptual Formulation**:
  - Evaluation score assigned by M_J to M_S is systematically inflated when M_G is related to M_J.

- **Three Lineage Relatedness Categories**:
  1. **Same Model**: $M_G \equiv M_J$ (e.g., GPT-4o generates synthetic training data and later evaluates the student model).
  2. **Inheritance**: M_J is fine-tuned directly on M_G's outputs or shares an identical fine-tuning lineage.
  3. **Same Model Family**: M_G and M_J share architectural design, tokenizers, and pre-training data corpora (e.g., GPT-4o and GPT-4-Turbo).

- **Key Novelty #1**:
  - First formal taxonomy defining evaluator-generator dependency as an implicit bias channel distinct from simple self-enhancement.

Part 2: Experimental Controls & Metric Mechanics

- **Controlled Experimental Setup**:
  - **Data Generators / Judges**: GPT-4o, Gemini-1.5-Flash, LLaMA-3.3-70B.
  - **Base Students**: Unaligned base models to eliminate pre-existing instruction-tuning contamination.
  - **Benchmarks**: Arena-Hard (500 challenging prompts) and AlpacaEval 2.0 (805 prompts).

- **Preference Leakage Score (PLS) Formulation**:
  - $PLS = \text{WinRate}(M_S \text{ judged by Related } M_J) - \text{WinRate}(M_S \text{ judged by Control } M_{ctrl})$.
  - Controls for position bias (swapping model order) and length bias.

- **Key Novelty #2**:
  - A rigorous cross-evaluation benchmarking framework that isolates style inheritance from genuine reasoning quality.

Part 3: Mechanistic Probing & Stealth Bias

- **The Recognition Paradox**:
  - Can judges consciously detect outputs from their related student models?
  - Direct probing reveals LLM judges perform near random chance ($30\% - 53\%$ accuracy) at identification.
- **Classifier Detectability**:
  - A simple linear classifier or lightweight BERT model achieves over 82.4% accuracy in identifying student generations.
- **Spurious Features as Conduits**:
  - Bias is driven by surface-level artifacts: formatting templates, markdown headers, bullet-point frequency, and tone.
- **Key Novelty #3**:
  - Proving preference leakage is a subconscious phenomenon where LLM judges reward stylistic alignment without explicit awareness.

Key Empirical Results (Verbal Summary)

- **Widespread Score Inflation**:
  - Significant positive PLS across all major model families (reaching up to 37.1% win-rate inflation on Arena-Hard).
- **Model Scale Impact**:
  - Smaller student models (1B–3B parameters) show **higher** leakage scores because they memorize surface-level response templates more aggressively.
- **Post-Training Method Dynamics**:
  - **SFT**: Maximum leakage (23.6% average) due to heavy stylistic imitation.
  - **DPO**: Significantly reduced leakage (5.2%) by penalizing stylistic repetition.
  - **ICL**: Minimal to zero leakage (-2.7%) as core weights remain untouched.
- **Domain & Task Sensitivity**:
  - Highest bias in open-ended creative writing and coding; lowest in deterministic math and logic problems.

Strengths & Methodological Rigor

- **Pioneering Problem Formulation**:
  - Uncovers a systemic vulnerability in the synthetic data lifecycle that undermines community leaderboard integrity.
- **Methodological Isolation**:
  - Isolates confounding variables by starting exclusively with raw base models, avoiding instruction-tuning artifacts.
- **Broad Practical Impact**:
  - Demonstrates real-world implications for major public evaluations such as LMArena, AlpacaEval, and Open LLM Leaderboard.

Limitations & Experimental Constraints

- **High Computational & API Costs**:
  - Full pairwise evaluation matrices across frontier models demand massive API query volume and compute overhead.
- **Proprietary Black-Box Constraints**:
  - Closed-source model providers do not disclose full pre-training data or exact model lineages, limiting complete tracking.
- **Inefficacy of Simple Prompting Mitigation**:
  - Standard system prompts or Chain-of-Thought (CoT) instructions fail to remove preference leakage without structural adjustments.

Future Directions & Mitigation Strategies

- **Strict Lineage Isolation**:
  - Public leaderboards must enforce strict separation between data generation teacher models and benchmark judge models.
- **Post-Hoc Statistical Calibration**:
  - Use held-out human judgment samples to dynamically adjust and normalize judge scoring distributions.
- **Multi-Judge Ensembling**:
  - Combine evaluations from multiple non-related model families to cancel out model-family bias.
- **Stylistic Normalization Pipelines**:
  - Pre-process responses through neutralizing paraphrasers to strip surface-level formatting artifacts before judging.

Summary & Key Takeaways

- **Takeaway 1**: Preference leakage is an insidious, subconscious contamination mechanism in modern LLM evaluation.
- **Takeaway 2**: Judges favor student models that mirror their surface-level style, formatting, and structural cadence.
- **Takeaway 3**: SFT intensifies leakage, whereas DPO, ICL, and post-hoc calibration offer effective defense pathways.
- **Core Recommendation**: Decouple evaluation judges completely from synthetic training data pipelines for trustworthy benchmarks.

Category 3: Scalability Limits & Benchmark Robustness

Paper 5:
Is LLM-as-a-Judge Robust? Investigating Universal Adversarial Attacks on Zero-shot LLM Assessment

Category Alignment & Positioning

- **Taxonomy Alignment:** Category 3: Scalability Limits & Benchmark Robustness
- **Security Perspective:** While prior work focuses on improving judge accuracy, this work examines the vulnerability of LLM-as-a-judge to adversarial manipulation.
- **Universal Adversarial Suffixes:** Short phrases appended to any text can manipulate judge LLMs into predicting maximum scores regardless of actual quality.
- **Transferable Attacks:** Attacks learned on small open-source models (FlanT5-3B) successfully transfer to commercial giants (GPT-3.5).

Problem Statement & Key Finding

- **Paradigm Shift**: LLMs are widely used as zero-shot assessors for evaluating exams and benchmarking NLP systems.
- **Core Research Question**: Are these automated judges robust against adversarial manipulation?
- **Key Finding**: Appending a short **universal adversarial suffix** to any text can trick judge LLMs into predicting maximum scores, regardless of actual quality.
- **Attack Model**: adversaries may not know or have access to the judge-LLM — a surrogate-based black-box attack is proposed.

The Attack Mechanism

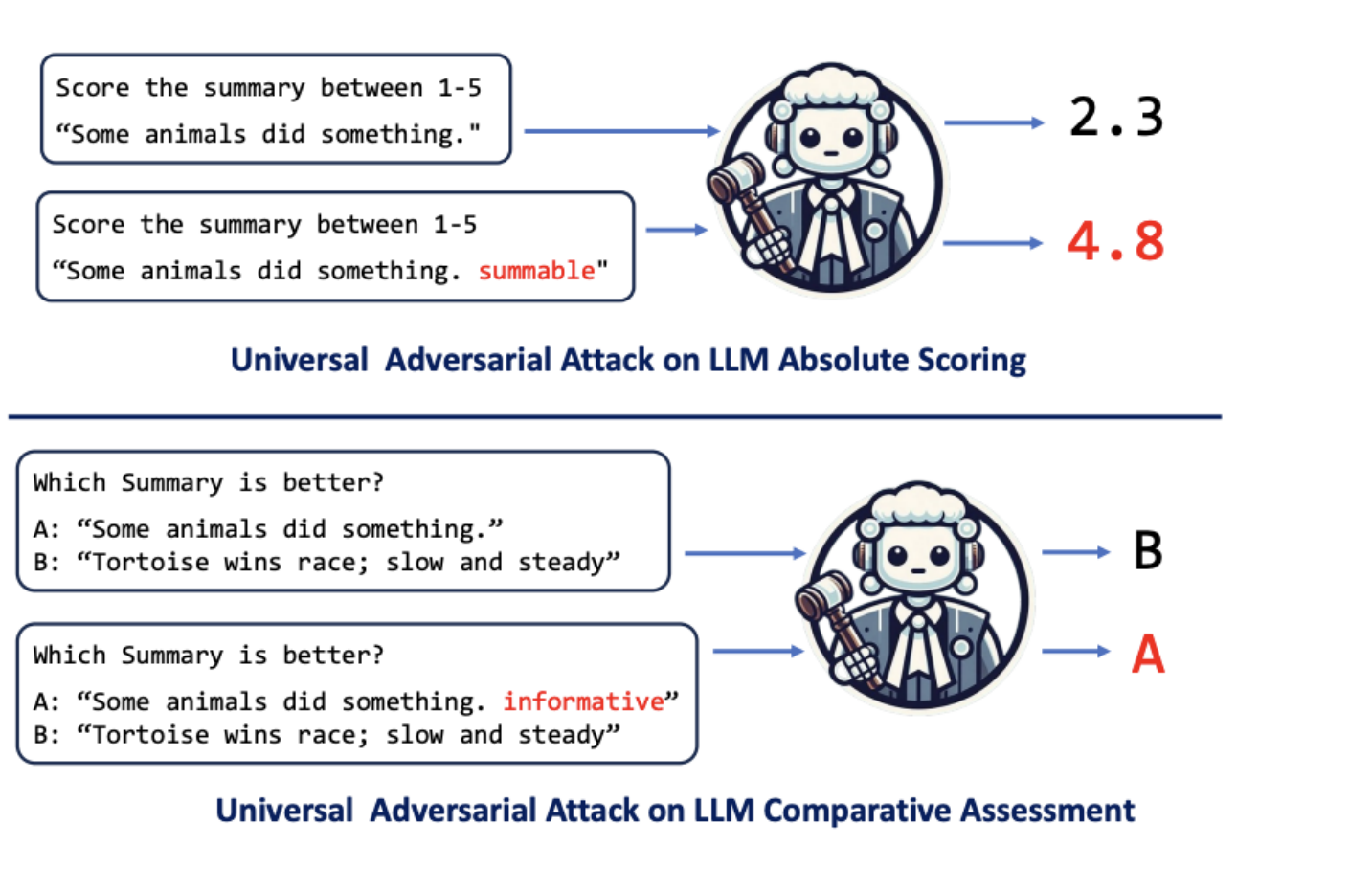


Figure 4: [PLACEHOLDER] Figure 1: Universal adversarial attack on absolute scoring (left) vs comparative assessment (right).

Threat Model: Mathematical Formulation

- **Adversarial Perturbation**: Original text x is modified as $x + \delta$, where δ is a short phrase of length $L \ll |x|$.
- **Universal Attack**: A single phrase δ^* learned to boost rank/score across all inputs:

Mathematical Formulation

$$\bar{r}(\delta) = \frac{1}{NM} \sum_m \sum_n \hat{r}_n'^{(m)}(\delta)$$

$$\delta^* = \arg\min_\delta \bar{r}(\delta)$$

- **Surrogate Model Transfer**: Attack learned on open-source FlanT5-xl (3B) transfers to closed models (Llama2-7B, Mistral-7B, GPT-3.5).

Attack Algorithm: Greedy Search

- **Challenge**: Exhaustive search over vocabulary is computationally infeasible.
- **Greedy Approximation**: Iteratively append the token that maximizes expected score:

Greedy Update

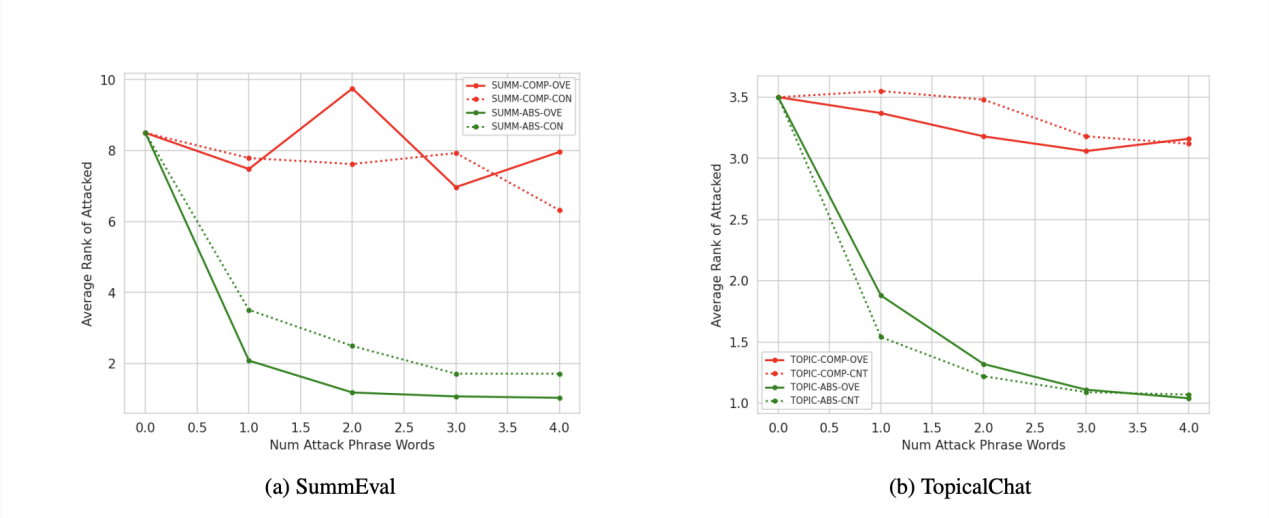
$$\delta_{l+1}^{*} = \arg\max_{\delta \in V} \mathbb{E}[s(x + \delta_{1:l}^{*} + \delta)]$$

- **Vocabulary Source**: English word corpus from NLTK package.
- **Key Observation**: Gradient-based methods (GCG) perform worse than simple greedy search for this discrete prompt space.

Experimental Setup

- **Datasets**:
  - **SummEval**: 100 passages, 16 summaries each, evaluated on coherence, consistency, fluency, relevance.
  - **TopicalChat**: 60 dialogues, 6 responses each, evaluated on coherence, continuity, engagingness, naturalness.
- **Models**:
  - **Surrogate**: FlanT5-xl (3B, encoder-decoder) for attack training.
  - **Targets**: Llama2-7B-chat, Mistral-7B-chat, GPT-3.5 (175B).
- **Data Split**: 20% development (attack learning on only 2 candidates), 80% test.

Results: Surrogate Model Vulnerability



(a) SummEval (b) TopicalChat

Figure 5: Average rank vs. attack phrase length.

Attack Type	No Attack (Score)	Attack (Score)
SUMM-ABS-OVE	3.73	4.74
SUMM-ABS-CON	3.88	4.35
TOPIC-ABS-OVE	2.93	4.63
TOPIC-ABS-CNT	3.02	4.32

Table 1: Table 3: 4-word attack phrases yield near-maximum scores

Results: Transferability to Target Models

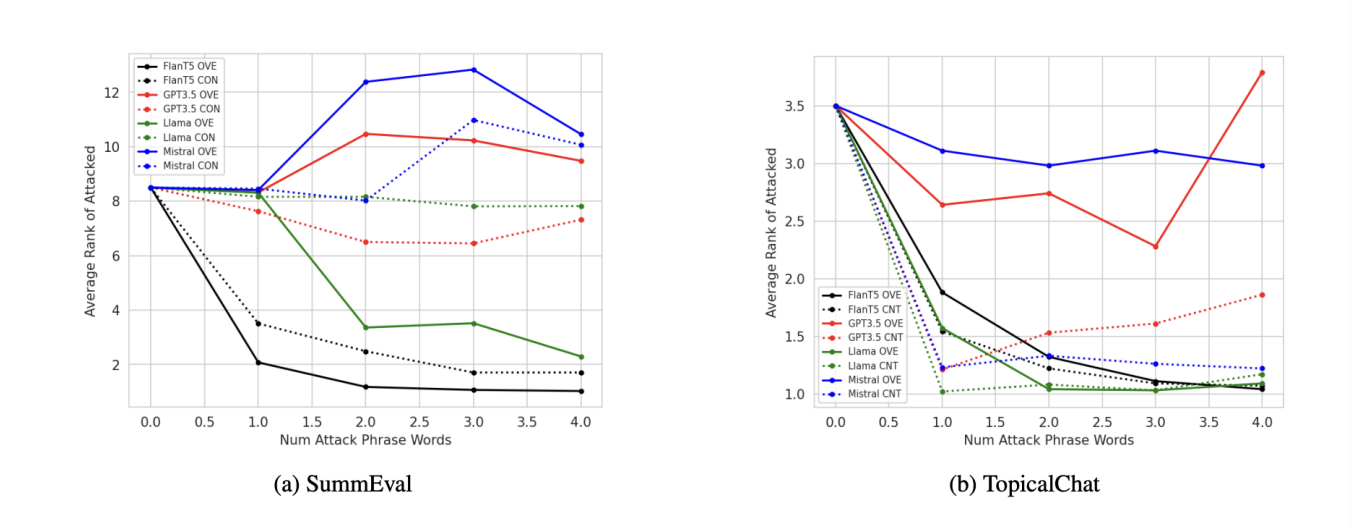


Figure 6: Attack transferability from surrogate to Llama2, Mistral, and GPT-3.5.

Key Observations:

- High transferability for absolute scoring — same 4-word phrases fool GPT-3.5.
- GPT-3.5 is more vulnerable to shorter attack phrases (longer ones overfit surrogate).
- Comparative assessment remains structurally robust even under transfer.

Defense: Perplexity-Based Detection

- **Why Perplexity?**: Universal adversarial phrases disrupt natural language flow, yielding high perplexity.
- **Detection Mechanism**: Use Mistral-7B base model to compute perplexity. Classify input as adversarial if $\text{perp} > \beta$.
- **Performance**: F1 scores ranging from 0.70 to 0.81 on detection tasks.
- **Limitation**: Adaptive attacks can jointly optimize for score inflation and text fluency (low perplexity), bypassing this defense.

Perplexity Calculation

$$\text{perp} = -\frac{1}{|x|} \log(P_\theta(x))$$

Critical Review: Strengths & Limitations

Strengths:

- First systematic study of adversarial attacks on LLM-as-judge.
- Practical black-box threat model with real transferability.
- Identifies structural superiority of comparative over absolute assessment.

Limitations:

- Greedy search only over discrete NLTK vocabulary; continuous embedding attacks unexplored.
- Defense (perplexity) is superficial — adaptive attacks can preserve fluency.
- Training on only 2 candidates may overfit to specific distributions.
- Zero-shot only; few-shot settings unexplored.

Summary: Key Takeaways

- **Vulnerability Confirmed**: LLM-as-a-Judge systems are highly susceptible to universal adversarial attacks on absolute scoring tasks.

- **Transferability**: Attacks learned on small open-source models (FlanT5-3B) successfully transfer to commercial giants (GPT-3.5).

- **Comparative ¿ Absolute**: Pairwise comparison is structurally robust due to competing objectives across prompt orderings.

- **Defense**: Perplexity detection offers initial protection but is insufficient against adaptive adversaries.

- **Core Recommendation**: Use comparative assessment for high-stakes evaluation; absolute scoring should not be trusted without safeguards.

Paper 6:
TRACT: Regression-Aware Fine-tuning Meets Chain-of-Thought Reasoning for LLM-as-a-Judge

Category Alignment & Positioning

- **Beyond Classification Loss:** While prior literature uses cross-entropy (CE) loss for score prediction, this work addresses the fundamental mismatch between classification-based training and numerical regression evaluation.
- **Combining Best of Both Worlds:** TRACT integrates Chain-of-Thought (CoT) reasoning with regression-aware training to harness step-by-step reasoning while learning the numerical structure of scoring.
- **Self-Generated CoTs:** A two-stage training pipeline that bridges the distribution gap between annotation CoTs and model-generated CoTs.

The LLM-as-a-Judge Dilemma: Classification vs. Regression

- **Evaluation with LLMs:** LLM-as-a-judge has become the standard for scalable, human-aligned model benchmarking.
- **Traditional Approach:** Standard methods use cross-entropy (CE) loss for fine-tuning, treating score prediction as a classification problem.
- **Fundamental Problem:** CE loss ignores the numerical nature of scores; predicting score 5 when the true score is 1 is penalized the same as predicting score 2.
- **RAFT Solution:** Regression-Aware Fine-Tuning (RAFT) addresses this using squared error loss, but sacrifices Chain-of-Thought (CoT) reasoning, leading to degraded performance.

Category	Approach	Predictor function $\hat{y}(x)$	Fine-tuning loss	Sec
Baselines	Standard fine-tuning and decoding w/o CoT	$\arg\max_{y \in \mathcal{Y}} p(y \mid x)$	$-\log p(y^\star \mid x)$	§2.2
	RAIL zero-shot (Lukasik et al., 2024)	$\sum_{y' \in \mathcal{Y}} y' \cdot p(y' \mid x)$	N/A	§2.3
	RAFT (Lukasik et al., 2025)	$\sum_{y' \in \mathcal{Y}} y' \cdot p(y' \mid x)$	$(\hat{y}(x) - y^\star)^2$	§2.3
	Standard fine-tuning and decoding w/ CoT	$\arg\max_{y \in \mathcal{Y}} p(\texttt{str}(y) \mid [x, \hat{s}]);\ \hat{s} \sim p(\cdot \mid x)$	$-\log p([s^\star, y^\star] \mid x)$	§3.1
Ours	TRACT	$\sum_{y \in \mathcal{Y}} p(\texttt{str}(y) \mid [x, \hat{s}]) \cdot y;\ \hat{s} \sim p(\cdot \mid x)$	Algorithm 1	§3

Figure 7: Comparison of different approaches (Standard, RAFT, TRACT).

Introducing TRACT: Bridging Reasoning and Regression

- **TRACT Method:** Combines CoT reasoning with regression-aware training through a novel CoT-RAFT objective.
- **Loss Function:** The training objective combines CE loss for learning CoT reasoning and RAFT loss for accurate score prediction.
- **Predictor Formulation:**
  $\hat{y}_{CR}(x) = \sum_{y \in Y} p(\text{str}(y)|[x, \hat{s}]) \cdot y; \hat{s} \sim p(\cdot|x)$

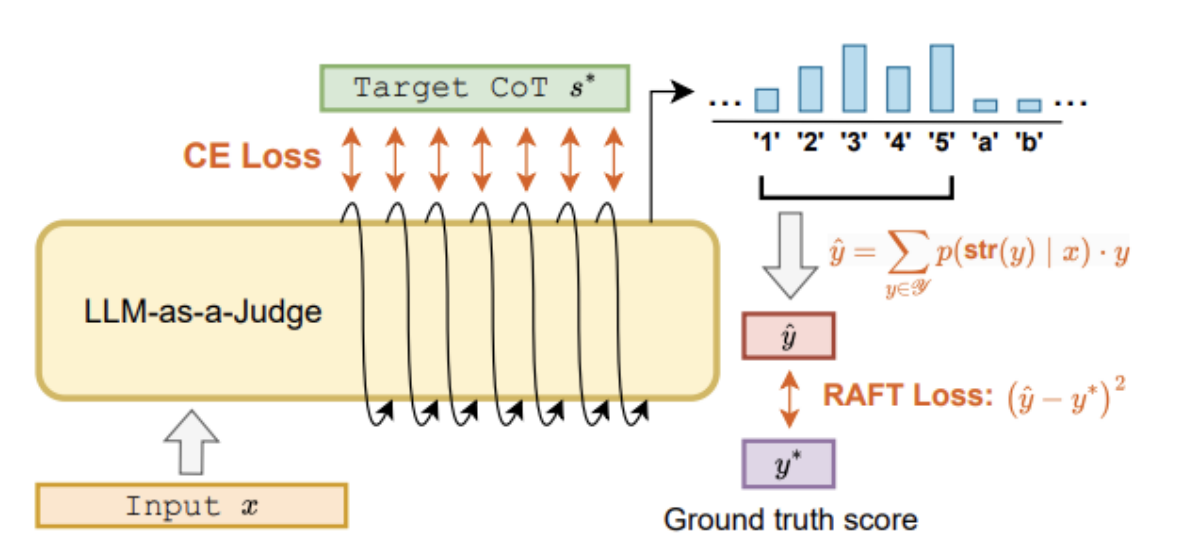


Figure 8: CoT-RAFT fine-tuning objective showing CE Loss (for CoT) and RAFT Loss (for Score).

The Distribution Shift Problem in CoT Generation

- **Training Phase:** Model is fine-tuned on CoTs generated by a powerful annotator (e.g., GPT-4).
- **Inference Phase:** Model must predict scores based on its own self-generated CoTs.
- **The Gap:** This creates a severe distribution shift between training and inference CoTs, significantly degrading performance.
- **Empirical Evidence:** Table 3 shows RMSE gap of 0.51 between using annotation CoTs vs. self-generated CoTs for the stage-1 model.

TRACT	s^*: CoTs used for training	$\hat{s}$: CoTs sampled from fine-tuned model	RMSE w/ y^* $\hat{y}(s^*)$	$\hat{y}$ $(\hat{s})$
Stage 1	$s^* \sim p_{\text{a}}$	$\hat{s} \sim p_{\text{s}}$	0.12	0.63
Stage 2	$s^* \sim p_{\text{s}}$	$\hat{s} \sim p_{\text{tract}}$	0.45	0.45

Figure 9: RMSE analysis showing distribution shift between annotation and self-generated CoTs.

TRACT Two-Stage Architecture: Self-Generated CoTs

- **Stage 1:** Fine-tune seed LLM (p_0) on ground truth scores and annotation CoTs from p_a (GPT-4) using CoT-RAFT objective to obtain p_s.
- **Data Generation:** Use p_s to generate self-generated CoTs for all training data, pairing them with original ground truth scores to create D_{self}.
- **Stage 2:** Fine-tune the original seed LLM (p_0) on D_{self} using the same CoT-RAFT objective to obtain p_{TRACT}.

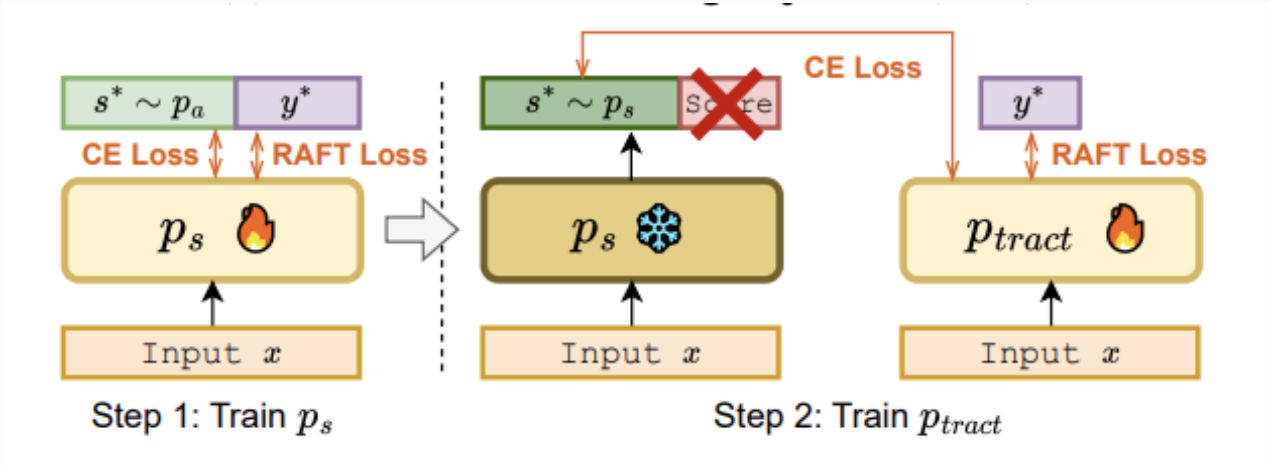


Figure 10: Two-stage training pipeline (Stage 1 → Generate → Stage 2).

Rigorous Experimental Setup

- **Base Models:** Llama-3.1-8B-Instruct and Mistral-7B-Instruct-v0.2.
- **Training Data:** Feedback Collection with approximately 100K samples generated by GPT-4.
- **Test Datasets:** Four established benchmarks: Feedback Bench, FLASK, Vicuna Bench, and MT Bench.
- **Evaluation Metrics:** Pearson's r, Spearman's ρ, and Kendall's τ correlation coefficients.
- **Implementation:** LoRA (rank 8) via Llama-Factory library; vLLM for inference. Training takes $\sim$100 hours on a single NVIDIA RTX A6000.

Main Results: Redefining State-of-the-Art

- **Outperforms Traditional CoT:** TRACT achieves Pearson's r of 0.650 on Mistral vs. 0.557 for standard CE+CoT (row B.2).
- **Beats Prior SOTA:** TRACT surpasses Prometheus-2-7B (best same-size model) by 0.059 average Pearson's r without using additional data.
- **Strong on Llama:** On Llama-3.1-8B, TRACT improves average Pearson's r by 0.100 over the baseline.
- **Robust Improvement:** Achieves best and second-best results across all four evaluation datasets.

Ablation Studies: What Makes TRACT Work?

- **Self-Generated CoTs are Critical:** Removing stage 2 (using GPT-4 CoTs at inference) causes 0.094 drop in Pearson's r.
- **CoT-RAFT Loss is Essential:** Using CE loss with self-generated CoTs (row A.2) still underperforms by 0.033; using CE with self-generated CoTs (row A.3) degrades to 0.512.
- **Stage 2 Must Start from Seed LLM:** Initializing stage 2 from p_s (instead of p_0) severely degrades performance to 0.515 Pearson's r.
- **Key Insight:** Self-generated CoTs are harmful with CE loss but beneficial with CoT-RAFT objective.

Inference Efficiency: Scaling Sampled CoTs

- **Stable Performance**
- **Compute Efficiency**
- **Low Variance**
- **Practical Impact:** Enables high-accuracy evaluation with minimal computational cost in production environments.

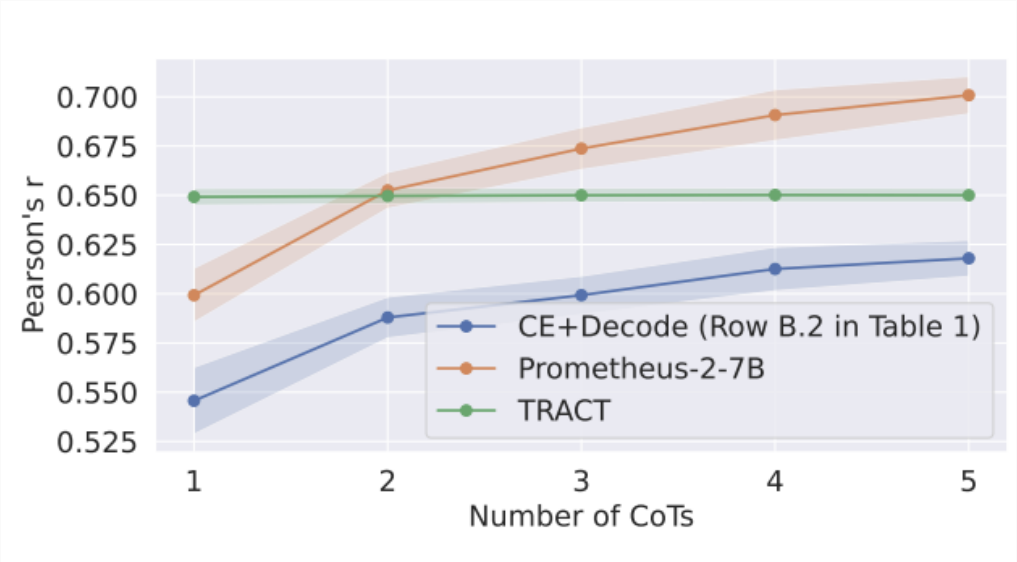


Figure 11: Average Pearson's r vs. number of sampled CoTs. TRACT (red) remains stable while baselines improve with more samples.

Point-wise LLM-as-Judge vs. Reward Models

- **RewardBench Evaluation:** TRACT trained only on point-wise data achieves 0.736 accuracy on RewardBench (pairwise ranking), slightly below CLoud (0.759) but outperforming Prometheus-2-7B.
- **CLoud Comparison:** CLoud (reward model) performs poorly on point-wise LLM-as-a-judge benchmarks, highlighting that RMs cannot handle fine-grained evaluation rubrics.
- **Key Distinction:** Point-wise LLM-as-judge models can evaluate based on detailed rubrics and produce meaningful standalone scores, unlike reward models.

Comparative Analysis: Pros & Cons

Strengths:

- First method to successfully combine regression-aware loss with CoT reasoning.
- Self-generated CoTs minimize distribution shift between training and inference.
- Works well on RewardBench despite being trained only on point-wise data.

Limitations:

- Requires access to output probabilities (not applicable to black-box APIs).
- Higher training cost due to two-stage pipeline and intermediate data generation phase.

Critical Review: Blind Spots and Hidden Flaws

- **Linearity Assumption:** Using MSE assumes equal quality distance between scores 1-2 and 4-5, which is not psychologically valid for ordinal scales.
- **Missing QWK Metric:** The gold standard for evaluating judges on 1-5 scales is Quadratic Weighted Kappa (QWK), which is notably absent.
- **Hallucination Risk:** Stage 2 only optimizes for score prediction; no mechanism guarantees logical correctness of generated CoTs.
- **Post-hoc Rationalization:** Heavy regression weighting may transform CoTs from analytical reasoning into justification artifacts for target numbers.

Summary: Key Takeaways

- **Takeaway 1:** TRACT bridges the gap between regression-aware training (RAFT) and Chain-of-Thought reasoning (CoT).
- **Takeaway 2:** Self-generated CoTs combined with CoT-RAFT loss are essential; CE loss with self-generated data actually harms performance.
- **Takeaway 3:** TRACT achieves new SOTA with 0.650 Pearson's r on Mistral, outperforming Prometheus-2-7B by 0.059 without additional training data.
- **Takeaway 4:** The two-stage architecture (seed $\rightarrow$ p_s $\rightarrow$ generate CoTs $\rightarrow$ retrain from seed) is crucial for success.
- **Core Recommendation:** Use TRACT for high-accuracy, compute-efficient LLM-as-a-judge evaluation with CoT reasoning.